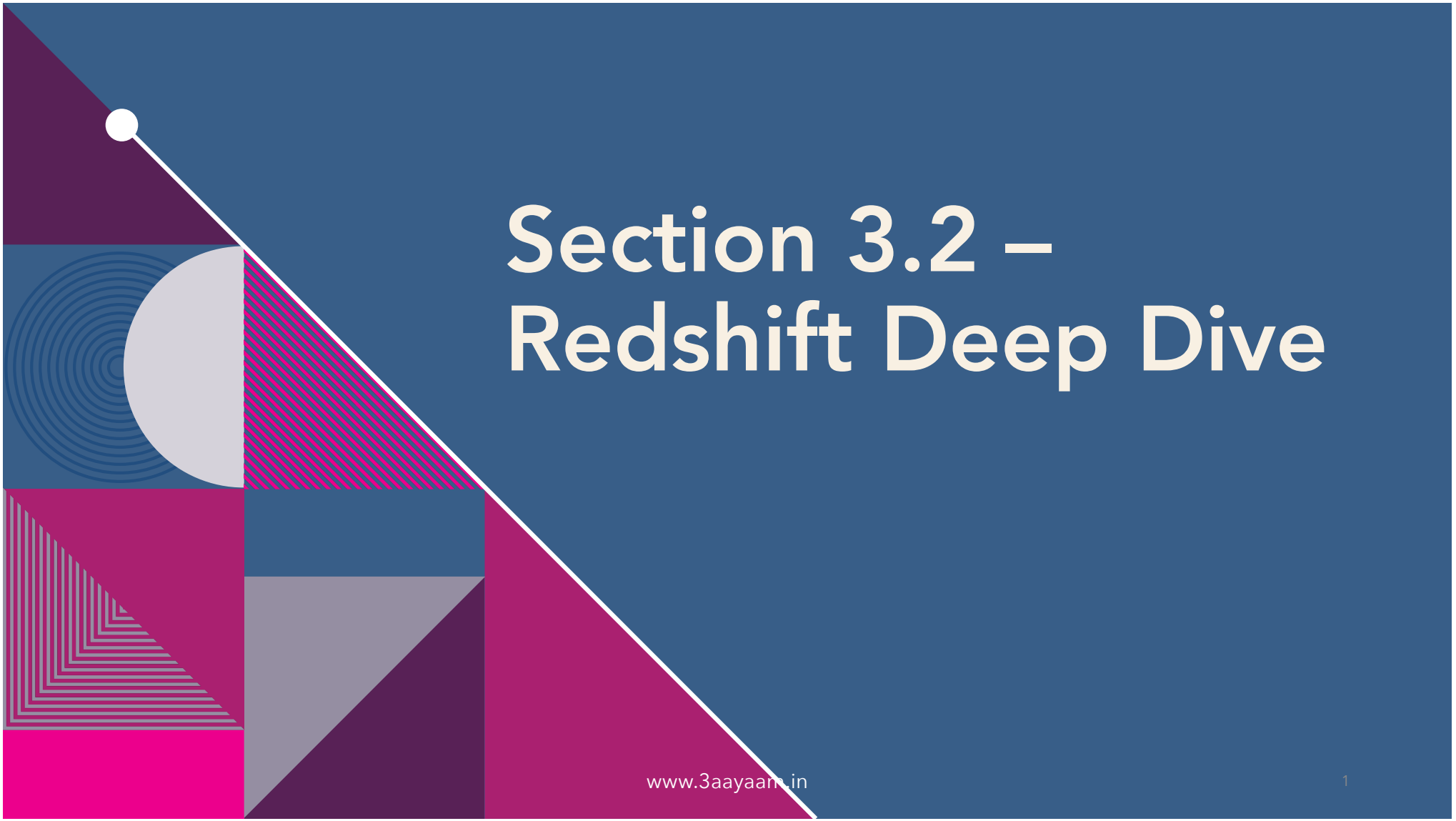
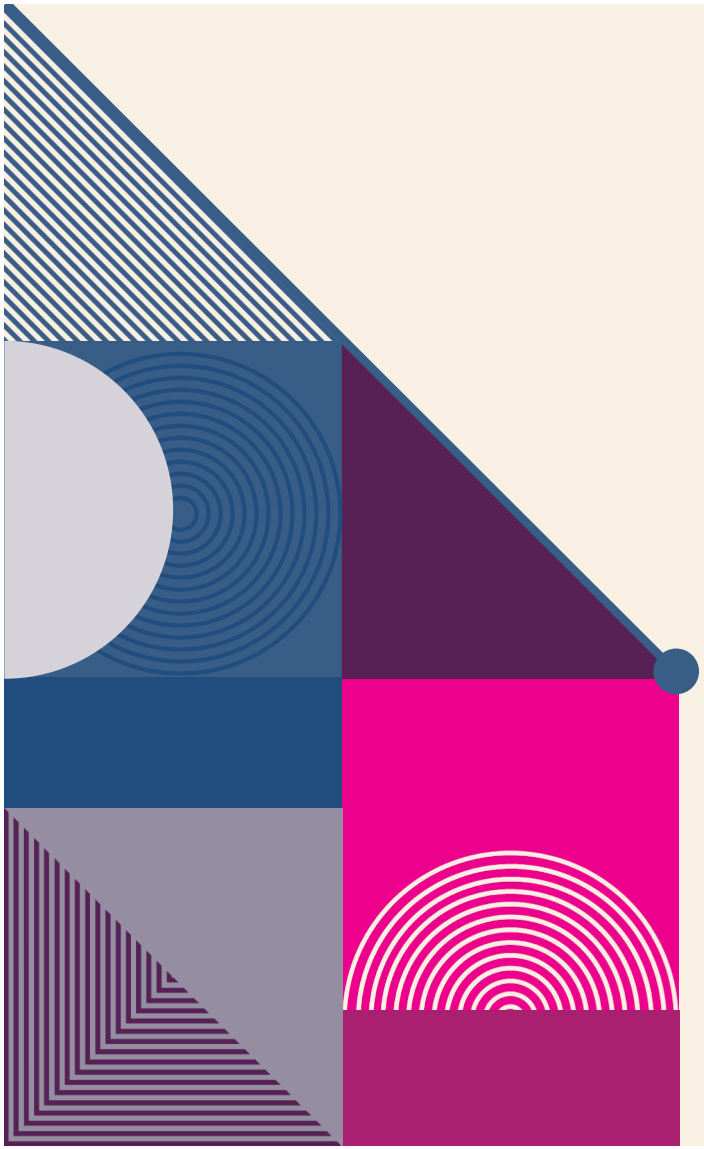

Section 3.2 –
Redshift Deep Dive



Agenda

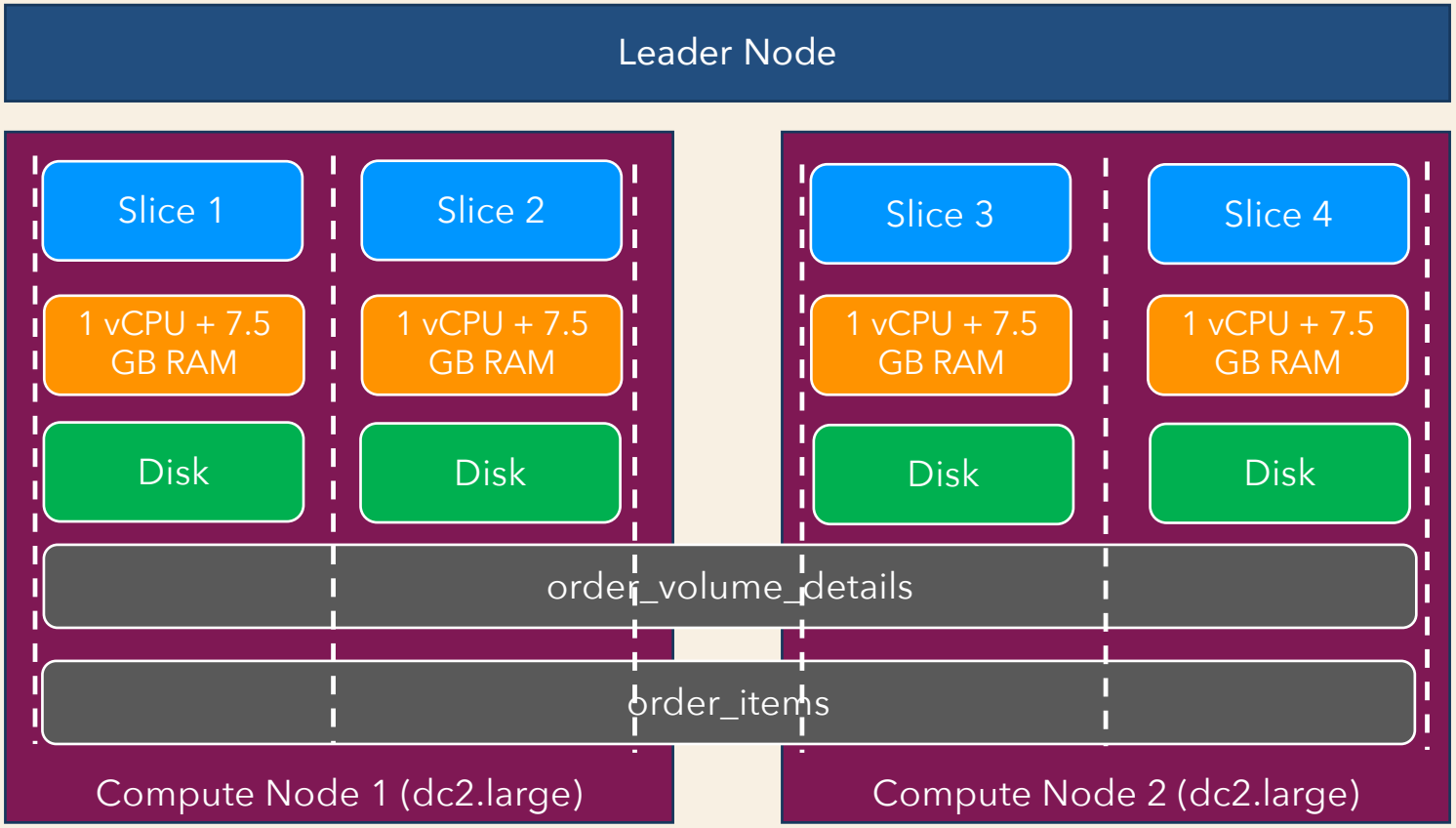
- Distribution and Sort Key
- Distribution Style
- Table Metadata
- Compression
- COPY & UNLOAD
- Load data from OLTP continuously
- Table maintenance

Distribution Key
Sort Key

RECAP: Table Creation

```
create table retail.order_items (
    order_id varchar(100),
    order_item_id int,
    product_id varchar(100),
    seller_id varchar(100),
    shipping_limit_date timestamp,
    price decimal(8,2),
    freight_value decimal(8,2),
    primary key (order_id),
    foreign key (product_id) references retail.products (product_id),
    foreign key (seller_id) references retail.sellers (seller_id)
);
```

Distribution Key



Distribution Key

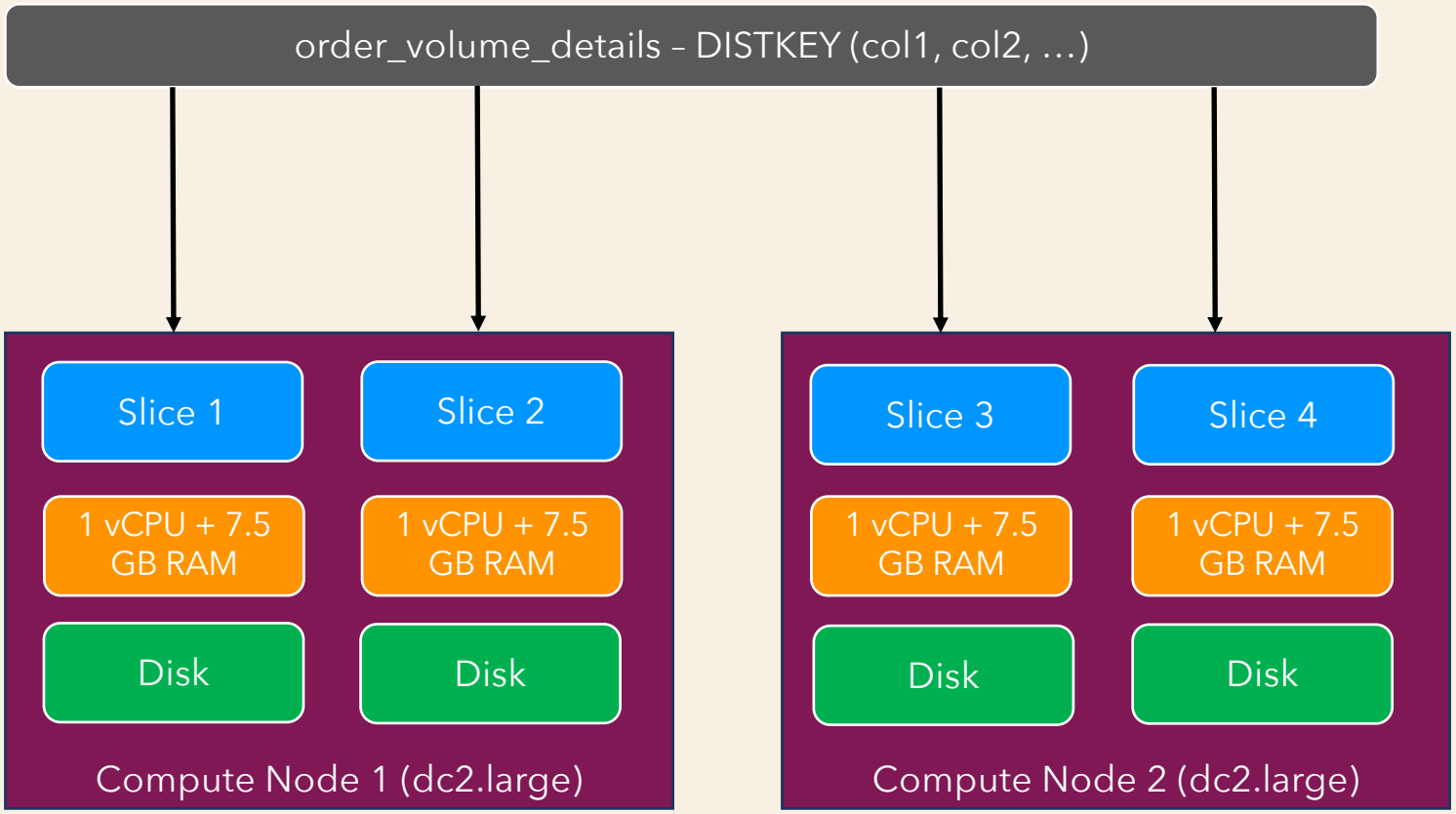


Table Creation

```
create table retail.order_items (
    order_id varchar(100),
    order_item_id int,
    product_id varchar(100),
    seller_id varchar(100),
    shipping_limit_date timestamp,
    price decimal(8,2),
    freight_value decimal(8,2),
    primary key (order_id),
    foreign key (product_id) references retail.products (product_id),
    foreign key (seller_id) references retail.sellers (seller_id)
    DISTKEY(order_id)

);
```

Distribution Style

- Defines how rows are distributed across the slices.
- EVEN – Distributes rows across slices using round-robin fashion.
- KEY – Using hashing of distribution column values.
- ALL – Entire table is distributed to all Nodes.
  - In the first slice of every node.
- AUTO – Redshift decides, based on size of table.

Table Creation

```
create table retail.order_items (
    order_id varchar(100),
    order_item_id int,
    product_id varchar(100),
    seller_id varchar(100),
    shipping_limit_date timestamp,
    price decimal(8,2),
    freight_value decimal(8,2),
    primary key (order_id),
    foreign key (product_id) references retail.products (product_id),
    foreign key (seller_id) references retail.sellers (seller_id)
    DISTKEY(order_id)
    DISTSTYLE even | key | all | auto

);
```

Distribution Key – which one??

- How to decide which column should be Distribution Key
- Understand query patterns
  - Columns with high cardinality
  - Columns with equality condition in WHERE clause
  - Join columns
  - Understand query patterns

Sort Key

- Defines how rows are physically sorted on disk.
- Compound Sort Key – With one or more columns
  - Prefix based queries
  - Joins, GROUP BY, ORDER BY, PARTITION BY
  - Regular INSERT, UPDATE, DELETE

```
WHERE order_id = <value>
```

```
WHERE order_id = <value>
AND   product_id = <value>
```

```
WHERE order_id = <value>
AND   product_id = <value>
AND   seller_id = <value>
```

```
Compound Sort Key
(order_id, product_id, seller_id)
In the same order
```

```
WHERE product_id = <value>
```

Sort Key

- Interleaved Sort Key – Equal weight to each column
  - Equal weightage to each column
  - Increased table load time
  - Increased table maintenance time
- AUTO – Redshift chooses

```
WHERE order_id = <value>
```

```
WHERE product_id = <value>
```

```
WHERE seller_id = <value>
```

```
Interleaved Sort Key
(order_id, product_id, seller_id)
In any order
```

Table Creation

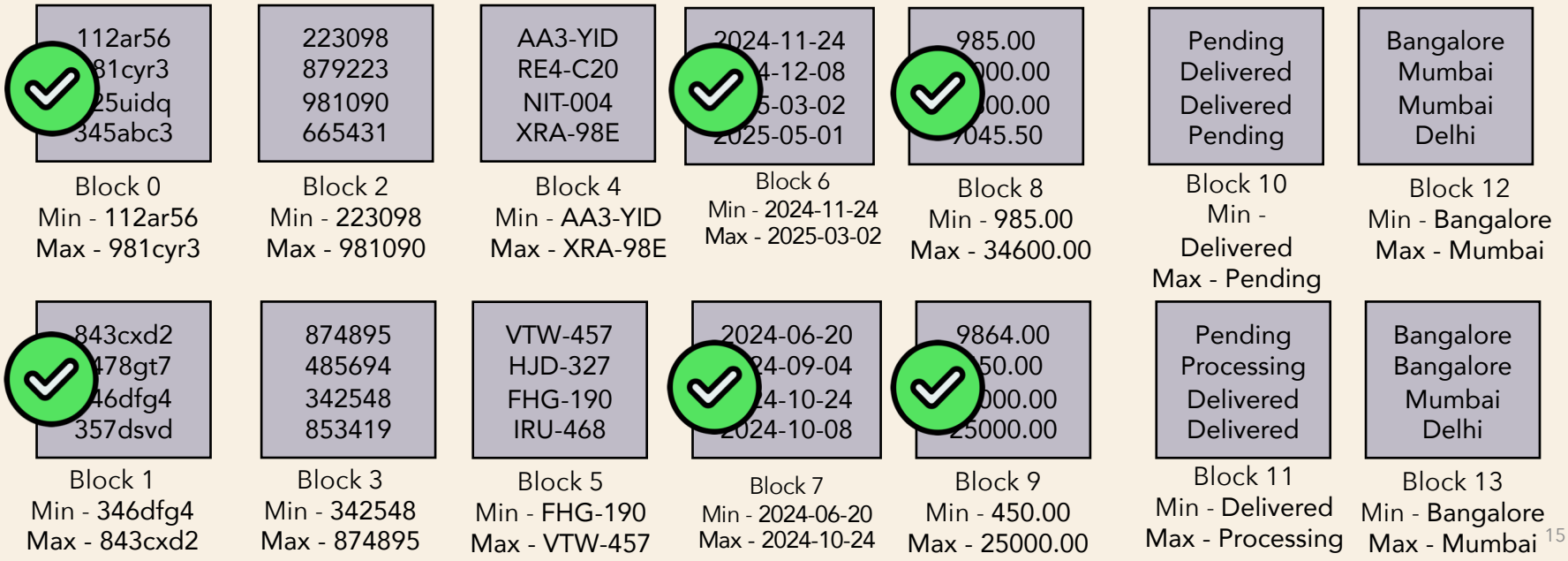
```
create table retail.order_items (
    order_id varchar(100),
    order_item_id int,
    product_id varchar(100),
    seller_id varchar(100),
    shipping_limit_date timestamp,
    price decimal(8,2),
    freight_value decimal(8,2),
    primary key (order_id),
    foreign key (product_id) references retail.products (product_id),
    foreign key (seller_id) references retail.sellers (seller_id)
    DISTKEY(order_id),
    DISTSTYLE KEY
    COMPOUND SORTKEY (order_id, product_id, seller_id)
    INTERLEAVED SORTKEY (order_id, shipping_limit_date)
);
```

Sort Key – which one??

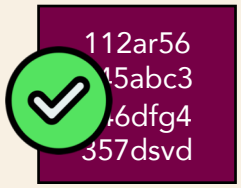
- How to decide which column should be Sort Key
- Understand query patterns
  - Columns in ORDER BY, GROUP BY clause
  - Columns in PARTITION BY clause Window function
  - Columns with equality condition in WHERE clause
  - Range queries
  - Join columns
  - Understand query patterns

Order_id	Customer_id	Seller_id	Order_date	Order_amount	Order_status	location
112ar56	223098	AA3-YID	2024-11-24	985.00	Pending	Bangalore
981cyr3						Mumbai
625uidq						Mumbai
345abc3						Delhi
843cxd2						Bangalore
5478gt7						Bangalore
346dfg4						Mumbai
357dsvd	853419	IRU-468	2024-01-08	25000.00	Delivered	Delhi

```
SELECT order_id, order_date, order_amount
FROM   order_items
WHERE  order_id = '346dfg4'
```



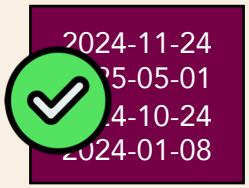
Order_id	Customer_id	Seller_id	Order_date	Order_amount	Order_status	location
112ar56	223098	AA3-YID	2024-11-24	985.00	Pending	Bangalore
345abc3	665431	XRA-98E	2025-05-01	9045.50	Pending	Delhi
346dfg4	342548	FHG-190	2024-10-24	10000.00	Delivered	Mumbai
357dsvd	853419	IRU-468	2024-01-08	25000.00	Delivered	Delhi
5478gt7	485694	HJD-327	2024-09-04	450.00	Processing	Bangalore
625uidq	981090	NIT-004	2025-03-02	34600.00	Delivered	Mumbai
843cxd2	874895	VTW-457	2024-06-20	9864.00	Pending	Bangalore
981cyr3	879223	RE4-C20	2024-12-08	11000.00	Delivered	Mumbai



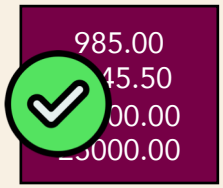
Block 0
Min - 112ar56
Max - 357dsvd

Block 2
Min - 223098
Max - 853419

Block 4
Min - AA3-YID
Max – IRU-468



Block 6
Min - 2024-11-24
Max - 2024-01-08



Block 8
Min - 985.00
Max - 25000.00

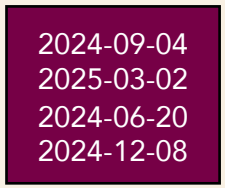
Block 10
Min - Delivered
Max - Pending

Block 12
Min - Bangalore
Max - Mumbai

Block 1
Min - 5478gt7
Max - 981cyr3

Block 3
Min - 485694
Max - 879223

Block 5
Min - HJD-327
Max - RE4-C20



Block 7
Min - 2024-09-04
Max - 2024-12-08

Block 9
Min - 450.00
Max - 11000.00

Block 11
Min - Delivered
Max - Processing

Block 13
Min - Bangalore
Max - Mumbai

Modify Distributed and Sort Key

- ALTER command

```
ALTER TABLE table_name
ALTER DISTKEY col_name
ALTER DISTSTYLE ALL | EVEN | KEY | AUTO
```

```
ALTER TABLE table_name
ALTER SORTKEY col_name
```

Compression

- At column level
- Reduces storage size
- Reduces I/O
- ENCODE

```
create table retail.order_items (
    order_id varchar(100) ENCODE RAW,
    order_item_id int ENCODE AZ64,
    product_id varchar(100) ENCODE BYTEDICT,
    seller_id varchar(100) ENCODE LZO,
    shipping_limit_date timestamp ENCODE ZSTD,
    price decimal(8,2) ENCODE MOSTLY8,
    freight_value decimal(8,2) ENCODE DELTA,
    order_details varchar(40) ENCODE TEXT255
    DISTKEY(order_id),
    DISTSTYLE KEY   ENCODE AUTO
);
```

Viewing Table Metadata Information

- SVV_TABLE_INFO – Shows detailed table information.
- SVV_ALL_TABLES – Shows all tables.
- SVV_DISK_USAGE – Shows data allocation for tables.
- STV_SLICES – Shows Slice information for a Node.
- STV_TBL_PERM – Provides table-wise Block and Slice information.
- STV_BLOCKLIST – No. of 1MB blocks per table.
- STV_PARTITIONS – Provides Node-wise disk distribution.
- SVL_AUTO_WORKER_ACTION – Whether Redshift performed an alter to change table distribution key
- SVV_ALTER_TABLE_RECOMMENDATIONS – Redshift recommendation for Distribution Key and Sort Key
- SVV_REDSHIFT_COLUMNS – Provides column level information like Encoding.

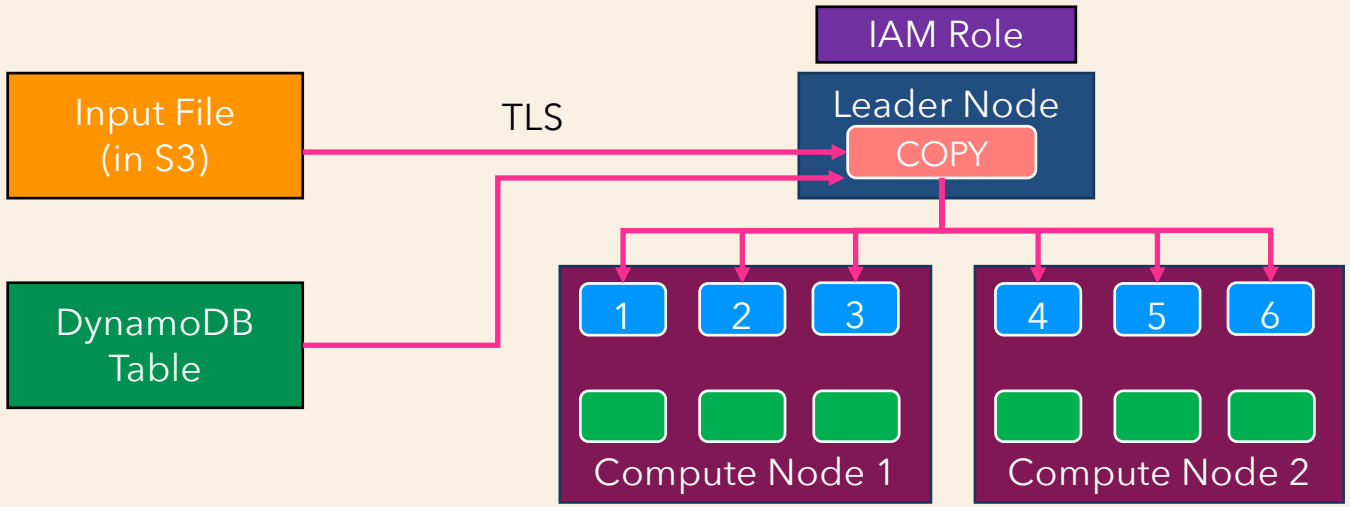
Hands-on

- Check table details using System Tables
- Alter table and check DISTKEY and SORTKEY
- Check how compression can save space
  - Load table with RAW encoding
  - Load the same table with manual encoding
  - Load the table again with auto encoding
  - Check the table size each time and query execution time
- Monitoring (CloudWatch)

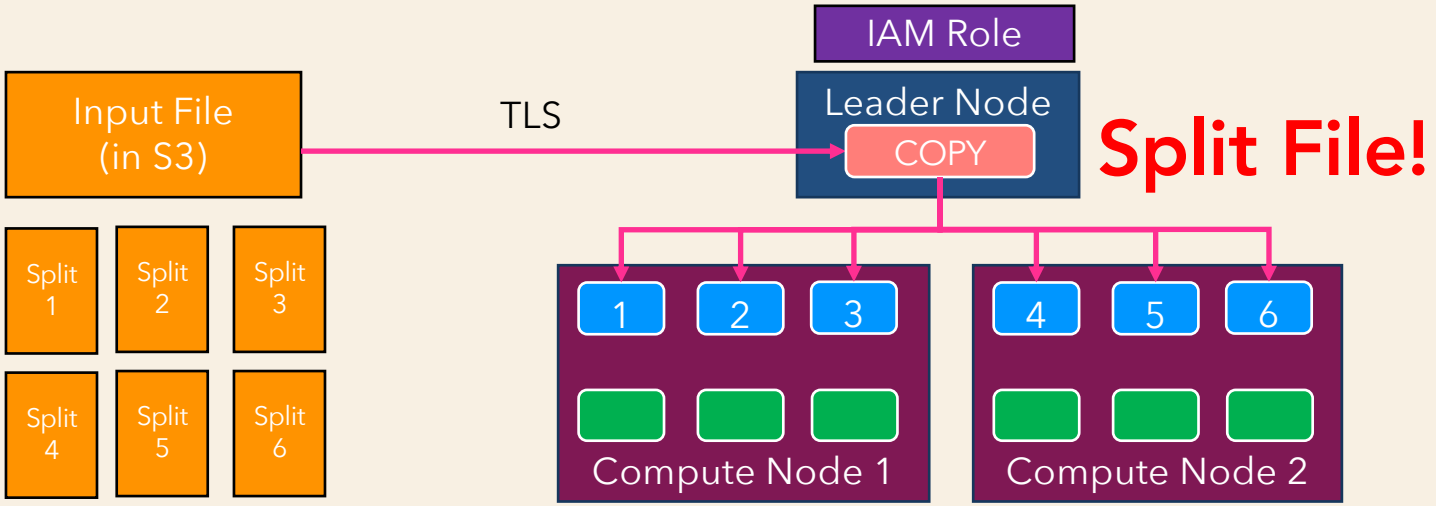
Load and Unload Tables

Load Data from S3 (COPY)

- Loads data in parallel leveraging Redshift's MPP architecture.
- Cluster needs to have access to S3 resources (objects/buckets)
- Uncompressed and compressed (gzip, lzop, bzip2)



Load Data from S3 (COPY)



- No need to run multiple COPY commands for parallelism
- Split the file beforehand

Options in COPY Command

```
COPY table-name (col1, col2, ...) FROM 's3://<bucket>/<folder>'
IAM_ROLE '<arn-for-iam-role>'
ACCESS_KEY_ID 'access-key-id' SECRET_ACCESS_KEY 'secret-access-key'
FORMAT CSV | PARQUET | JSON | ORC | AVRO | FIXEDWIDTH
BZIP2 | GZIP | LZOP | ZSTD
TIMEFORMAT 'MM-DD-YY HH:MI:SS'
DATEFORMAT 'DD-MM-YYYY'
IGNOREHEADER <number_rows>
```

Load Data from S3 (COPY)

- SYS_LOAD_DETAIL
- STL_LOAD_ERRORS
- SYS_LOAD_ERROR_DETAIL
- SYS_LOAD_HISTORY
- STL_LOAD_COMMITS
- STL_LOAD_ERRORS
- STL_LOADERROR_DETAIL

Hands-on

- IAM Role
- Load “orders” using a single file (9.1 GB)
- Load “orders” again after splitting the file (1.2 GB per file)
- Check system tables for Load/COPY
- Various options in COPY commands

Load Data Using Manifest File

Input File 1
(S3 Bucket 1 / Folder 1)

Input File 2
(S3 Bucket 2 / Folder 1)

Input File 3
(S3 Bucket 2 / Folder 2)

Input File 4
(S3 Bucket 1 / Folder 2)

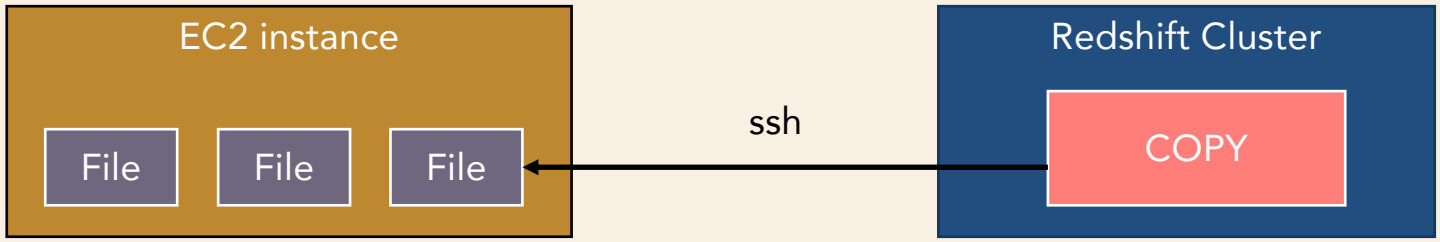
Manifest File

```
{ "entries": [
            {"url":"s3://bucket1/folder1", "mandatory":true},
            {"url":"s3://bucket2/folder1", "mandatory":true},
            {"url":"s3://bucket2/folder2", "mandatory":true},
            {"url":"s3://bucket1/folder2", "mandatory":true}
] }
```

```
COPY … FROM 's3://bucket/manifest-file'
IAM_ROLE '<your-iam-role>'
MANIFEST;
```

Load Data from LOCAL File

- From EC2



Hands-on

Load Data using INSERT & MERGE

- INSERT
- MERGE

Unload Data to S3 (UNLOAD)

- Extract record from a table and store it in S3
- The output rows are split across multiple files based on no. of slices.
- Parallel operation based on the no. of slices in the cluster.
- SYS_UNLOAD_DETAIL
- SYS_UNLOAD_HISTORY
- STL_UNLOAD_LOG

Unload Data to S3 (UNLOAD)

```
UNLOAD ('select-statement') TO 's3://<bucket>/<prefix>'
IAM_ROLE '<arn-of-iam-role>'
FORMAT CSV | PARQUET | JSON
MANIFEST
HEADER, DELIMITER (for CSV output format)
BZIP2 | GZIP | ZSTD
PARALLEL ON | OFF
MAXFILESIZE <size> MB | GB
ROWGROUPSIZE <size> MB | GB (for PARQUERT output format)
```

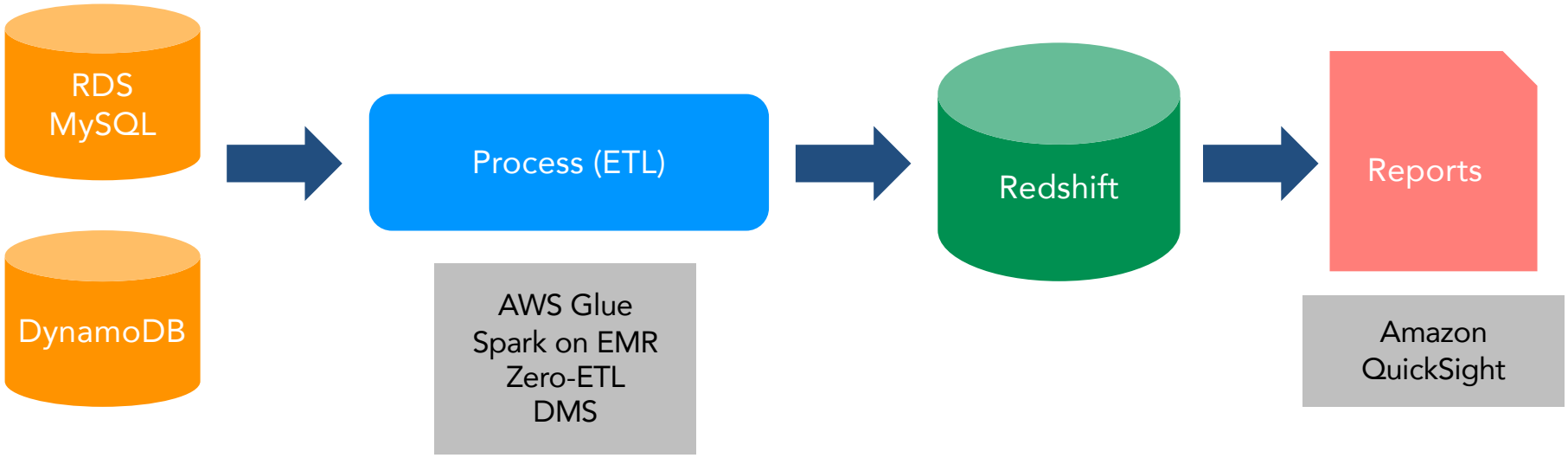
Hands-on

- Extract records from “orders” table in PARQUET format.
- Extract record with MANIFEST file.
- Use PARALLEL and MAXFILESIZE option.

Load Redshift from OLTP & Application using DMS

OLTP to OLAP

- ETL (Extract, Transform and Load)
- Processing (transformation)

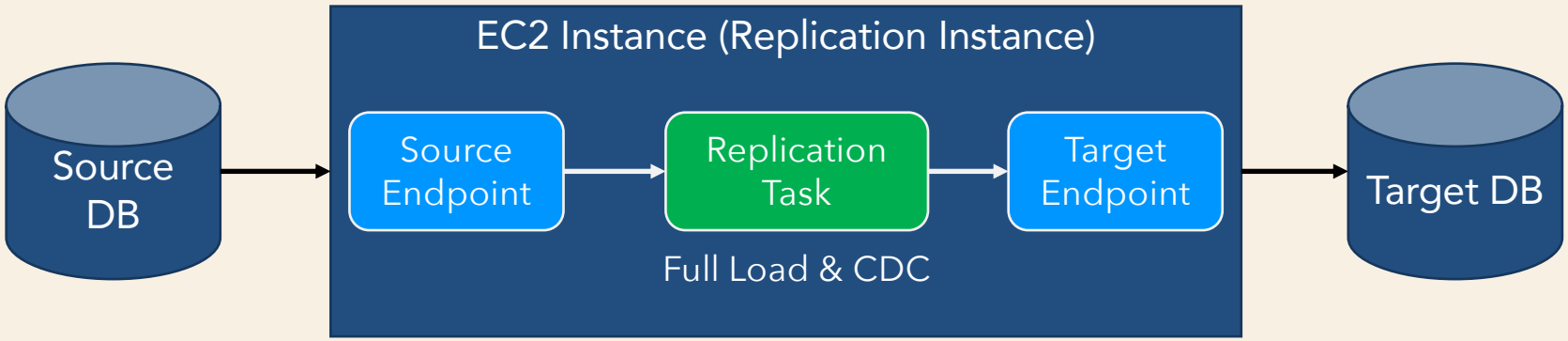


DMS (Database Migration Service)

- Migrate OLTP relational databases, DWH, NoSQL databases.
- Can be used to migrate from on-premise to AWS or from AWS to AWS, from other cloud to AWS.s

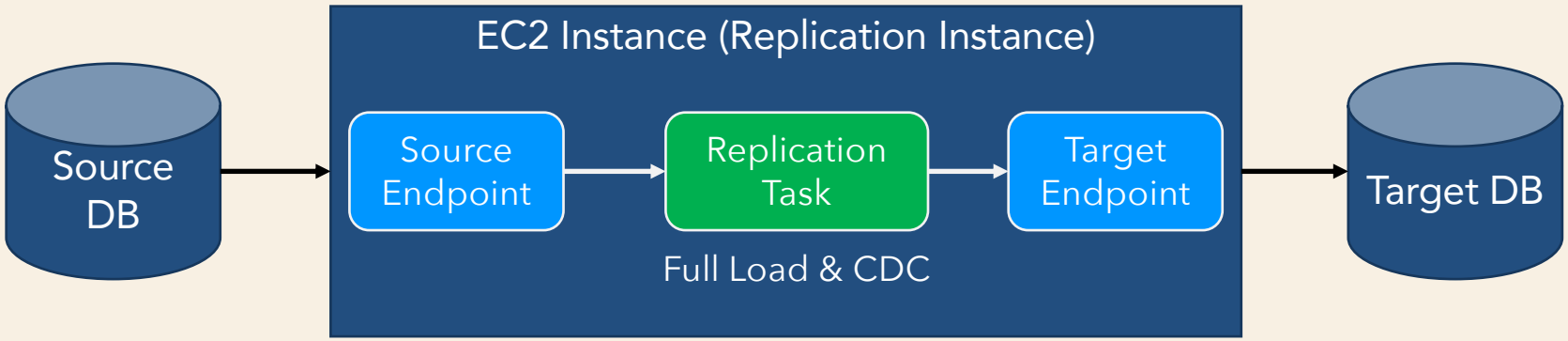
- Full Load – move existing data from source to target.
- CDC – move all data changes from source to target continuously.

DMS (Database Migration Service)



- Replication Task performs all the operation.
- Either Source or Target endpoint should be an AWS service
- SOURCES – Oracle, SQL Server, MySQL, MariaDB, DB2 LUW, PostgreSQL, MongoDB, Azure (SQL, MySQL, PostgreSQL), GCP (MySQL, PostgreSQL), OCI MySQL Heatwave

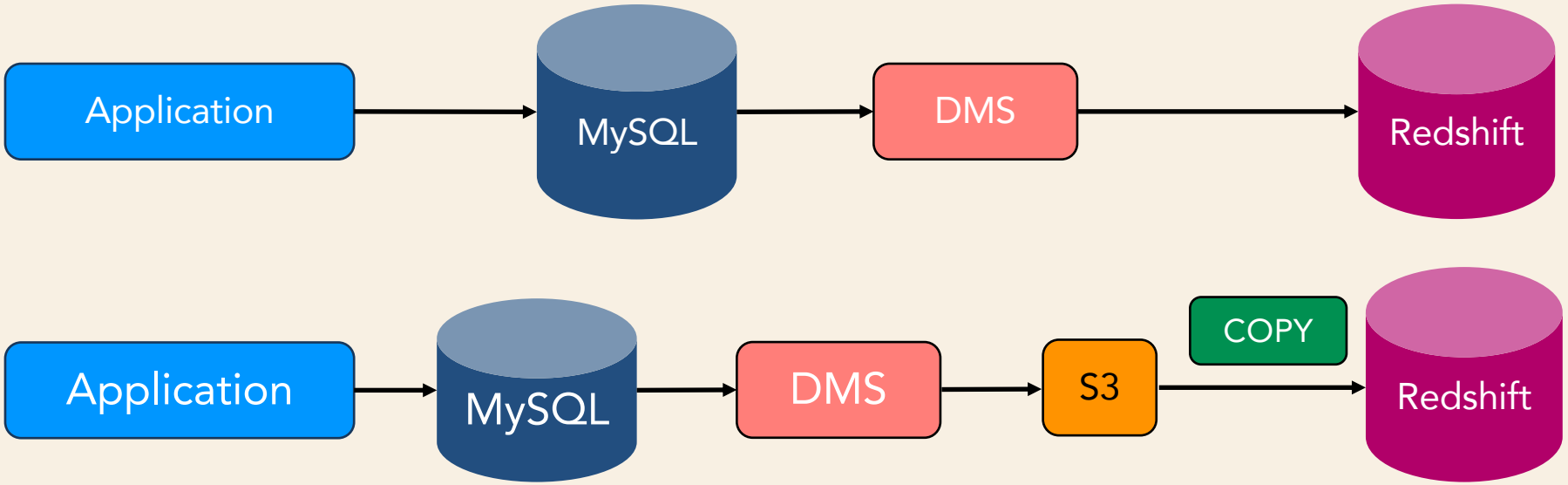
DMS (Database Migration Service)



- TARGETS – Oracle, SQL Server, MySQL, MariaDB, Redis OSS, Redshift, DynamoDB, S3, OpenSearch, ElastiCache, Kinesis Data Streams, DocumentDB, Kafka

DMS (Database Migration Service)

- OLTP sources
- Direct movement (without transformation)



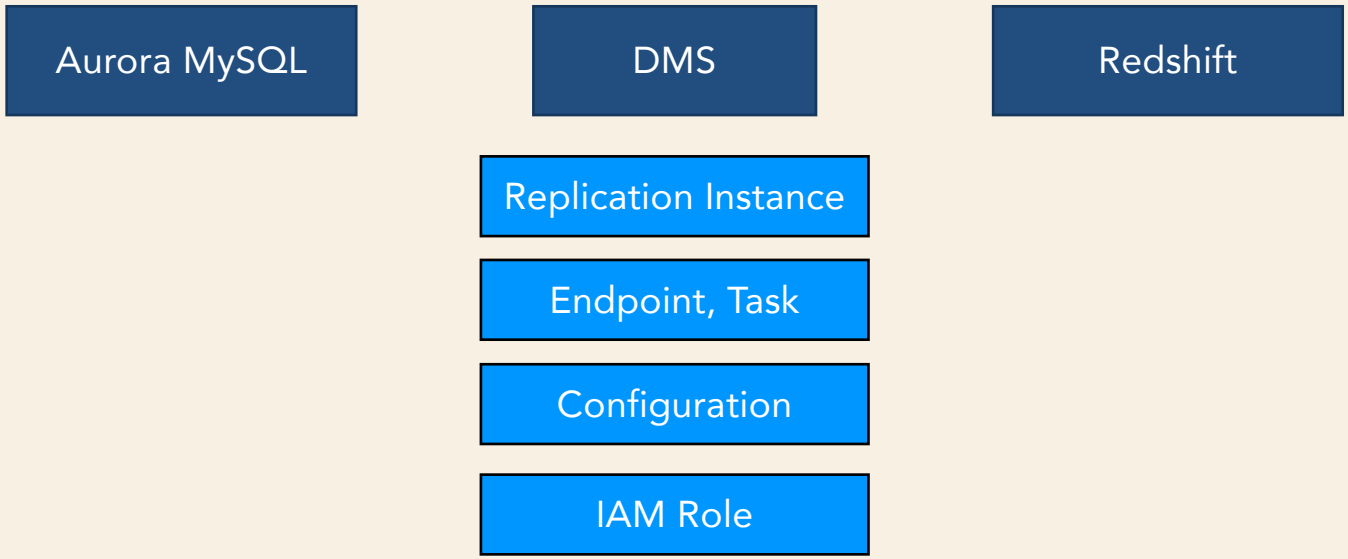
Hands-on

- Create Aurora MySQL data source.
- Create orders table in MySQL.
- Provision a DMS cluster and set up Aurora MySQL as source and Redshift as target.
- Use Python application to write to orders table in MySQL database.
- Check and monitor Redshift movement.
- CloudWatch metrics –
  - CDCLatencySource & CDCLatencyTarget
  - CDCChangesMemorySource & CDCChangesMemoryTarget
  - CDCChangesDiskSource & CDCChangesDiskTarget
  - CDCThroughputRowsSource & CDCThroughputRowsTarget

Load Redshift from OLTP using Zero-ETL

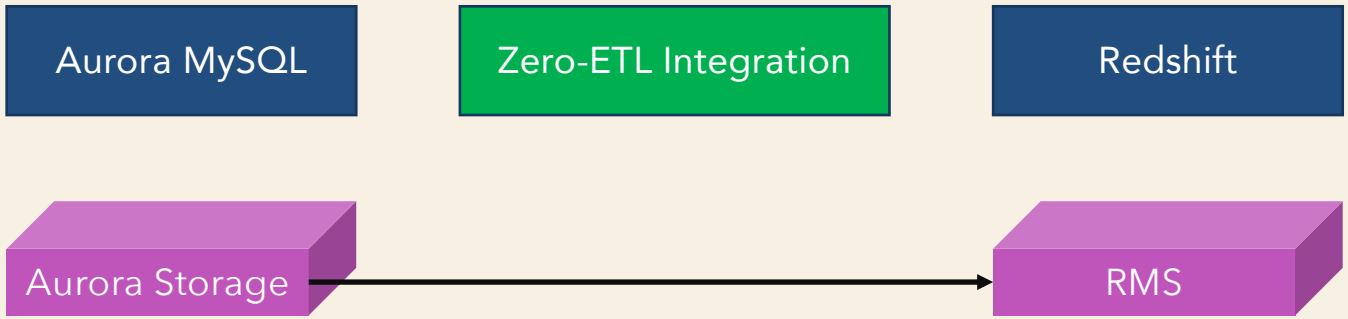
Zero-ETL

- OLTP sources
- Direct movement



Zero-ETL

- OLTP sources
- Direct movement



Zero-ETL Considerations

- RA3 node types
- Has to be encrypted
- Enabling case sensitivity
- Source and Target tables should have Primary Key defined
- Data filtering
- No transformation supported

Zero ETL Monitoring

- SVV_INTEGRATION – Provides configuration details.
- SYS_INTEGRATION_ACTIVITY – Info about completed integration runs.
- SVV_INTEGRATION_TABLE_STATE – Table level integration information.
- SYS_INTEGRATION_TABLE_STATE_CHANGE – Table state change log.
- CloudWatch metrics – IntegrationLag, IntegrationNumTablesReplicated, IntegrationNumTablesFailedReplication.

Hands-on

- Convert node types - dc2.large to ra3.xlplus
- Enable Encryption in the cluster.
- Enable Case Sensitivity.
- Create Zero-ETL integration in source Aurora MySQL.
- Create destination database in Redshift.
- Use Python application to write to Aurora MySQL database.
- Check and monitor Redshift movement.

Load Redshift from Application Directly

Python Connector

- Python library to connect to Redshift
- Install – **`pip3 install redshift_connector`**
- Import – **`import redshift_connector`**
- Make database connection
- Open cursor
- Execute query using cursor

```
conn = redshift_connector.connect(
       host='<redshift-endpoint>',
       port=5439,
       database='three_aayaam_db',
       user='admin',
       password='my_password'
)

cursor = conn.cursor()

query_output = cursor.execute("select * from
orders limit 10")
```

Data API

- No database drivers
- No connections
- No libraries etc.
- Use HTTP endpoint to run SQL statements without connections. Integration with AWS SDKs.
- Asynchronous calls.
- Max active queries - 500

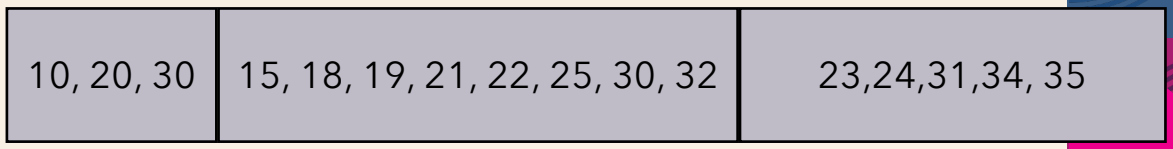
Deep Copy

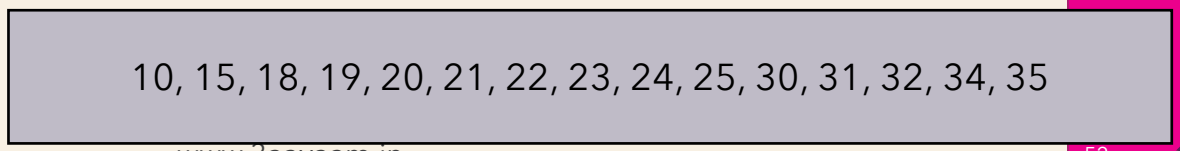
- Recreates tables
- Repopulates tables using bulk INSERT.
- Sorts the table automatically
- CREATE TABLE … INSERT INTO … SELECT * …
- CREATE TABLE LIKE
  - New table contains the same DISTKEY, SORTKEY and Encoding Scheme
  - Primary Key and Sort Key have to be created separately
- CTAS to create temp table and load the main table after truncating data.

Redshift Table Maintenance

VACUUM

- DELETE
- UPDATE
- VACUUM - Reclaim Space and Sorts table data





VACUUM

- DELETE
- UPDATE
- VACUUM - Reclaim Space and Sorts table data
- svv_table_info
  - vacuum_sort_benefit
  - unsorted
- IO intensive operation
- Takes longer for tables with INTERLEAVED sort keys

SORT Phase
Sorts rows in the unsorted region

MERGE Phase
Merges sorted rows with existing rows at the end of table

VACUUM

```
VACUUM FULL | SORT ONLY | DELETE ONLY | REINDEX
<table-name> TO <n> PERCENT
```

Note: For tables with Interleaved Sort Keys REINDEX has to be run separately

- Skips SORT phase if “unsorted” is less than 5.
- Automatic VACUUM DELETE ONLY
- Automatic VACUUM SORT

ANALYZE

- Updates the statistics so that query planner chooses optimal plan. Uses a sample of rows from tables for DISTKEY and all the other columns.
- Redshift runs it automatically in the background after monitoring changes in the workload (auto_analyse).
- COPY command runs ANALYZE automatically at the end after loading an empty table.
- New tables created with CTAS, SELECT INTO
- PG_STATISTIC_INDICATOR
- STL_ANALYZE

ANALYZE

```
ANALYZE <table-name> (<col1>, <col2>, ...)
PREDICATE COLUMNS | ALL COLUMNS

ANALYZE COMPRESSION <table> (<col1>, ...) COMPROWS <n>
```

- Resource intensive operation.
- Columns used in WHERE clause as filters, JOINs and GROUP BY clause should be considered.
- When to run manually?
  - After COPY commands or large UPDATE, INSERT and DELETEs
  - Before running queries
  - Use PREDICATE COLUMNS

Hands-on

- Load “orders” table using COPY command
- Check table size (svv_table_info)
- Perform UPDATE operation. Check table size (svv_table_info)
- Perform DELETE operation. Check table size (svv_table_info)
- Check unsorted rows using svv_table_info
- Run VACUUM DELETE ONLY
- Run VACUUM FULL
- Run ANALYZE on the “orders” table.
- Run ANALYZE COMPRESSION on “orders” table

SUMMARY

- Distribution Key & Sort Key, Compression
- System Tables
- Load data using COPY command and MANIFEST file
- Load data from LOCAL file in EC2
- Extract data from tables using UNLOAD command
- Move data from OLTP to DWH using DMS and Zero-ETL
- Python Connector, Data API, Deep Copy
- Table Maintenance using VACUUM and ANALYZE


THANK YOU
www.3aayaam.in